

TECHNICAL & FUNCTIONAL DOCUMENTATION

Claims Copilot

Agentic Prescriptive Analytics for Health Insurance Claims Review

System type: Full-stack web application (Angular 21 + FastAPI + Claude)

Domain: Health insurance claims adjudication, human-in-the-loop

Document version: 1.0

Prepared: 2026-09-11

Table of Contents

1. Executive Summary	3
2. System Architecture	3
3. Design Principles (Why the AI Never Decides)	4
4. Backend — Module Reference	5
5. The AI Review Pipeline — End-to-End Flow	7
6. Deterministic Business Rules (Tools)	9
7. Human Decision Workflow	10
8. REST API Reference	11
9. Data Model	12
10. Frontend — Application Structure	13
11. Audit Trail & Provenance	14
12. Configuration & Running the Application	14

1. Executive Summary

Claims Copilot is a demo application that shows how an AI agent (Claude) can safely sit inside a regulated decision workflow — health insurance claims adjudication — without ever being the one who makes the decision. Claude's role is strictly *orchestration and explanation*: it decides which verification/compliance/calculation checks to run and in what order, and afterwards writes a plain-English rationale. Every fact, every dollar (rupee) amount, and the final recommendation are produced by ordinary, deterministic Python business logic that Claude cannot see or influence directly.

The system consists of an **Angular 21** single-page frontend used by claims operations staff, and a **FastAPI (Python)** backend that exposes a small REST API, holds an in-memory store of applicant records, executes the four business-rule "tools," and optionally calls the Anthropic Claude API to orchestrate those tools and write a rationale.

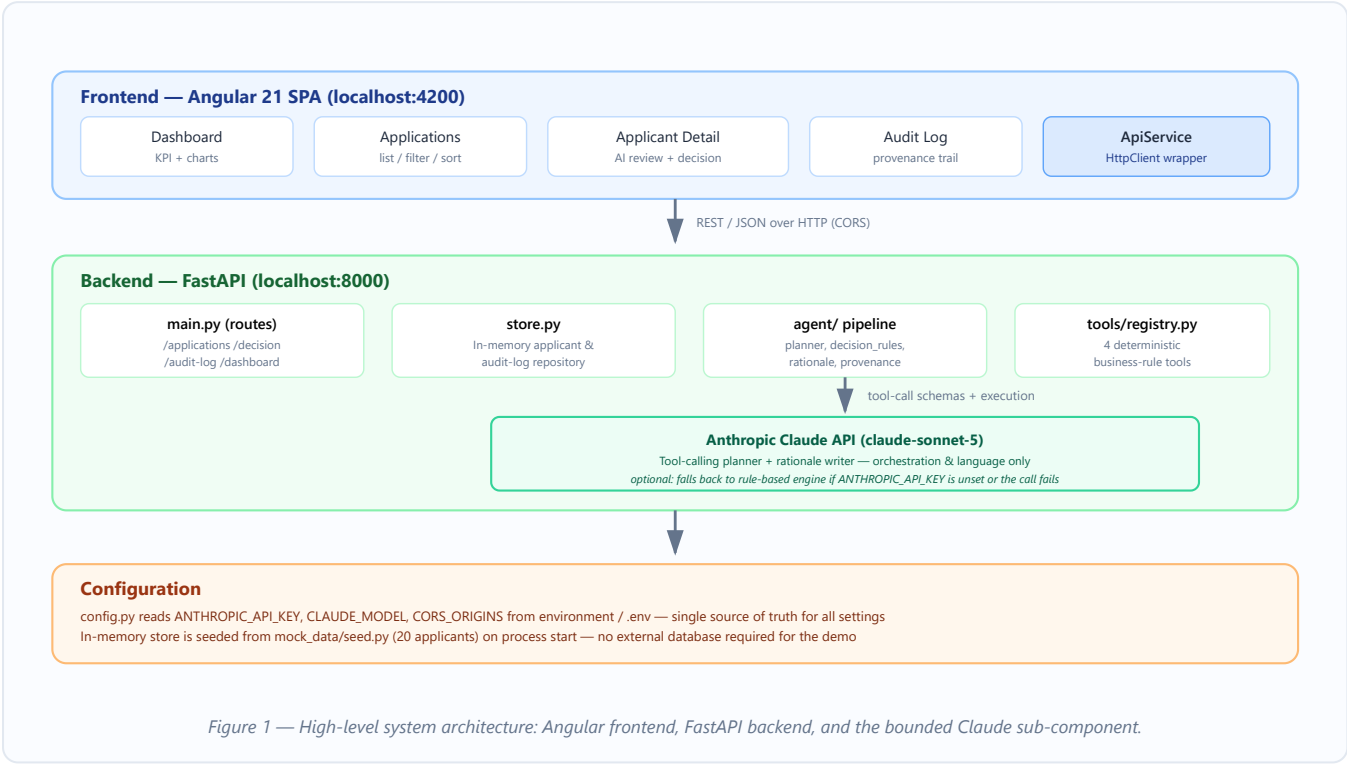
Core guarantee: The recommendation and approved amount returned to a reviewer are always computed by `app/agent/decision_rules.py` — a pure function with no AI involvement — regardless of whether Claude, or the deterministic fallback engine, decided which tools to call and in what order. A human reviewer is the only actor in the system who can set a claim to **approved** or **rejected**.

1.1 Key Capabilities

Capability	Description
Claims dashboard	KPI summary — total applications, pending count, approvals/rejections today, average processing time, total value approved, and a status breakdown chart.
Applications list	Filterable and sortable list of all claim applications by status, risk score, claim amount, tenure, etc.
Applicant detail + AI review	Full applicant profile with a one-click "Run AI Review" action that triggers the agent pipeline and shows the recommendation, amount, rationale, and the tool calls that produced them.
Human decision	Reviewer approves, rejects, or requests more information — optionally overriding the AI-suggested amount before approving.
Audit log	Full provenance trail of every tool invocation (inputs/outputs/timestamp) and every human decision, filterable by applicant.

2. System Architecture

The application follows a classic three-tier shape, with the AI positioned as a bounded sub-component of the middle tier rather than a peer service with direct access to data or decisions.



2.1 Technology Stack



3. Design Principles — Why the AI Never Decides

The codebase enforces a strict separation between *orchestration/language* (Claude) and *computation/decision* (Python) at every layer. This is the single most important architectural idea in the system and is documented directly in the module docstrings.

Principle	Where it's enforced
Claude never sees raw applicant data directly. It must call a tool to learn anything about an applicant.	<code>agent/planner.py</code> — Claude's tool call arguments are ignored; every tool always looks up the trusted server-side record by <code>applicant_id</code> .
Claude never computes the recommendation or the approved amount.	<code>agent/decision_rules.py</code> — a pure, deterministic function is the only place a <code>Decision</code> is produced, from tool outputs alone.
Claude cannot alter a decision when writing the rationale.	<code>agent/rationale.py</code> — the rationale prompt is given the decision as an already-fixed fact and instructed never to introduce a number it wasn't given.
The AI review can never itself approve or reject a claim.	<code>store.apply_decision()</code> is the only function that can set status to <code>approved</code> / <code>rejected</code> , and it is only reachable from the human-submitted <code>POST /decision</code> endpoint.
Every automated action is auditable.	<code>agent/provenance.py</code> 's <code>@traced</code> decorator wraps all four tools so every invocation (inputs, outputs, timestamp) is logged, regardless of which engine triggered it.
The system degrades gracefully without Claude.	If <code>ANTHROPIC_API_KEY</code> is unset, or any Claude call fails, the pipeline transparently falls back to a fixed-order rule-based engine and a templated rationale — the demo (and, in principle, production traffic) never blocks on the AI provider.

Auditability note: The `AIRecommendation.engine` field (`"claude"` or `"fallback_rules"`) is returned to the frontend and stored with every recommendation, so a reviewer always knows whether a given rationale was AI-written or templated.

4. Backend — Module Reference

`backend/app/` is organized by responsibility. Nothing outside `store.py` touches applicant data directly, and nothing outside `tools/registry.py` performs a business-rule computation.

4.1 Directory Layout

```

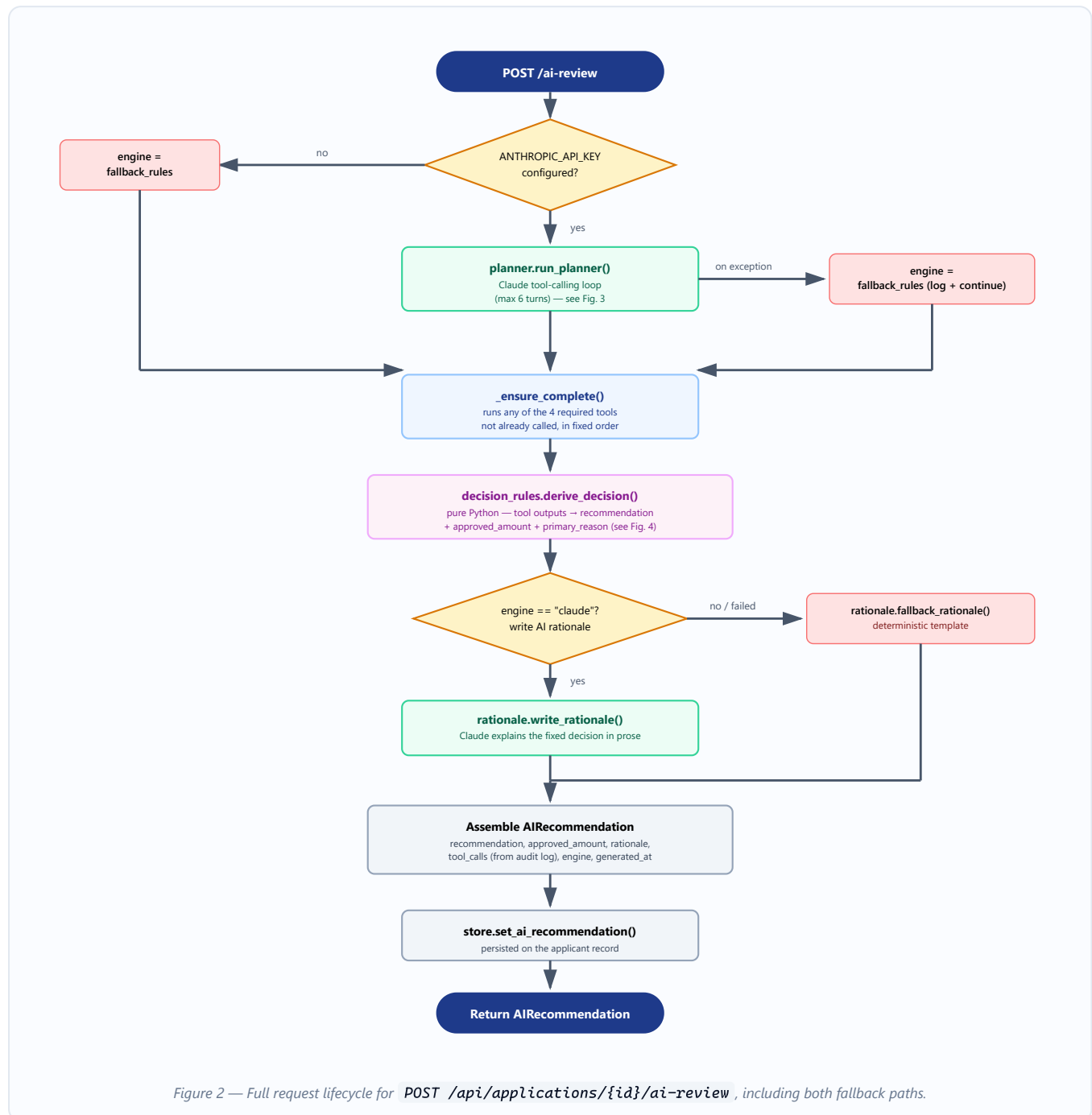
backend/
├── app/
│   ├── main.py           FastAPI app + all HTTP routes
│   ├── config.py         Env-driven configuration (single source of truth)
│   ├── store.py          In-memory applicant + audit-log repository
│   └── agent/
│       ├── planner.py     Claude tool-calling orchestration loop
│       ├── decision_rules.py Pure function: tool outputs → recommendation
│       ├── rationale.py   Claude / templated rationale writer
│       ├── provenance.py  @traced decorator → audit log
│       └── pipeline.py    Wires planner + rules + rationale together
│   ├── tools/
│       └── registry.py    4 deterministic business-rule tools + Claude schemas
│   ├── models/
│       └── schemas.py     Pydantic models (API contract)
│   └── mock_data/
│       └── seed.py        20 seed applicants for the in-memory store
├── requirements.txt
└── .env.example
  
```

4.2 Module Responsibilities

Module	Responsibility
<code>main.py</code>	Declares the FastAPI app, CORS policy, and 6 REST endpoints. Delegates all logic to <code>store</code> and <code>pipeline</code> ; contains no business logic itself.
<code>config.py</code>	Loads <code>ANTHROPIC_API_KEY</code> , <code>CLAUDE_MODEL</code> , and <code>CORS_ORIGINS</code> from environment/ <code>.env</code> . The single place that calls <code>os.getenv</code> .
<code>store.py</code>	Thread-safe, in-memory applicant + audit-log repository. Provides list/get/filter/sort, applies human decisions, and computes dashboard KPIs on demand.
<code>agent/planner.py</code>	Runs a bounded (max 6 turns) Claude tool-calling loop that decides which of the 4 tools to call and in what order for one applicant.
<code>agent/decision_rules.py</code>	Pure function <code>derive_decision()</code> mapping tool outputs to <code>{recommendation, approved_amount, primary_reason}</code> . No AI, no side effects.
<code>agent/rationale.py</code>	<code>write_rationale()</code> (Claude) and <code>fallback_rationale()</code> (templated) — both produce prose describing an already-fixed decision.
<code>agent/provenance.py</code>	<code>@traced</code> decorator: wraps a tool function so every call is written to the audit log automatically.
<code>agent/pipeline.py</code>	<code>run_ai_review()</code> — the single entry point that ties planner → fallback-completion → decision rules → rationale → persistence together.
<code>tools/registry.py</code>	The 4 business-rule tool functions plus their Claude JSON tool schemas. The only place real computation happens.
<code>models/schemas.py</code>	Pydantic models defining every request/response shape; mirrored field-for-field by the Angular TypeScript interfaces.
<code>mock_data/seed.py</code>	Builds 20 realistic seed applicants spanning every status and risk profile, with timestamps relative to process start.

5. The AI Review Pipeline — End-to-End Flow

Triggered by `POST /api/applications/{id}/ai-review` and implemented in `agent/pipeline.run_ai_review()`. This is the central workflow of the system.



5.1 Detail: Claude Planner Tool-Calling Loop

`agent/planner.run_planner()` runs up to `MAX_PLANNER_TURNS = 6` conversation turns. On each turn, Claude may request one or more tool calls; the backend executes each requested tool *against the trusted applicant_id* (ignoring any other arguments Claude supplied), records the result, and feeds the tool outputs back to Claude. The loop ends naturally when Claude stops requesting tools and instead replies with a short confirmation sentence — it is instructed never to state a recommendation or amount in that reply.

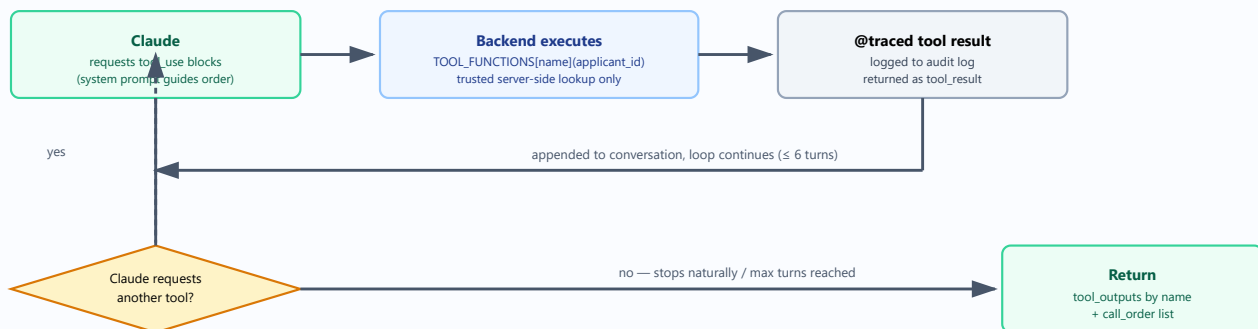


Figure 3 — Claude planner tool-calling loop (`agent/planner.py`). Claude only ever chooses which tool to call next; the tool's output is the sole source of every fact used downstream.

5.1.1 Planner System Prompt Guidance

1. Always call `verify_identity` first.
2. If identity verification fails, remaining checks may be skipped — nothing left to safely act on.
3. Otherwise call `check_fraud_flags` and `check_policy_compliance`.
4. Only call `calculate_risk_adjusted_amount` once identity has been verified.
5. Once assessment is complete, stop calling tools and reply with a short confirmation — never a number or recommendation.

If Claude is unavailable, times out, or raises an exception, `pipeline.run_ai_review()` catches it, logs it, and sets `engine = "fallback_rules"`. In that mode `_ensure_complete()` alone becomes the orchestrator: it calls the same four tools in the exact same fixed order described above, guaranteeing identical business coverage either way.

6. Deterministic Business Rules (Tools)

All four tools live in `tools/registry.py`, are wrapped with `@traced`, and take only `applicant_id` as input — every fact is looked up server-side, never trusted from the caller.

6.1 verify_identity

Confirms Aadhar and PAN identity documents. Checks that the on-file `*_verified` flag is `true` and the document matches its expected format (Aadhar: XXXX-XXXX-9999, PAN: AAAAA9999A). Returns `aadhar_valid`, `pan_valid`, plus the two raw format checks.

6.2 check_fraud_flags

Flags a claim if any of the following are true:

Rule	Threshold
Multi-claim + high amount pattern	≥ 3 prior claims and claim amount $> ₹200,000$
High risk score	<code>risk_score</code> ≥ 0.80
Large early-tenure claim	claim amount $\geq ₹450,000$ and policy tenure ≤ 1 year
Unverified identity documents	Aadhar or PAN not verified on file

6.3 check_policy_compliance

Rule	Requirement
Minimum waiting period	Policy tenure ≥ 1 year
Maternity claims	Policy tenure ≥ 2 years
Policy sub-limit	Claim amount $\leq ₹500,000$

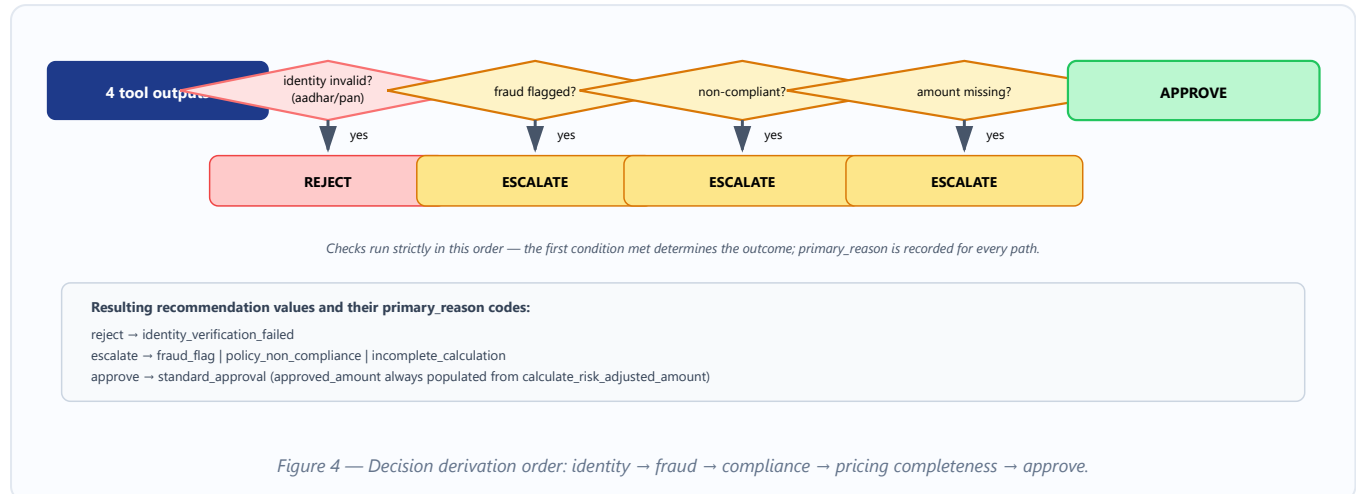
6.4 calculate_risk_adjusted_amount

The sole source of the approved amount. Computes an `approval_fraction` applied to the requested amount, rounded to the nearest ₹100:

```
tenure_bonus      = min(tenure_years, 5) × 0.02
risk_penalty      = risk_score × 0.50
prior_claims_penalty = min(prior_claims, 5) × 0.03

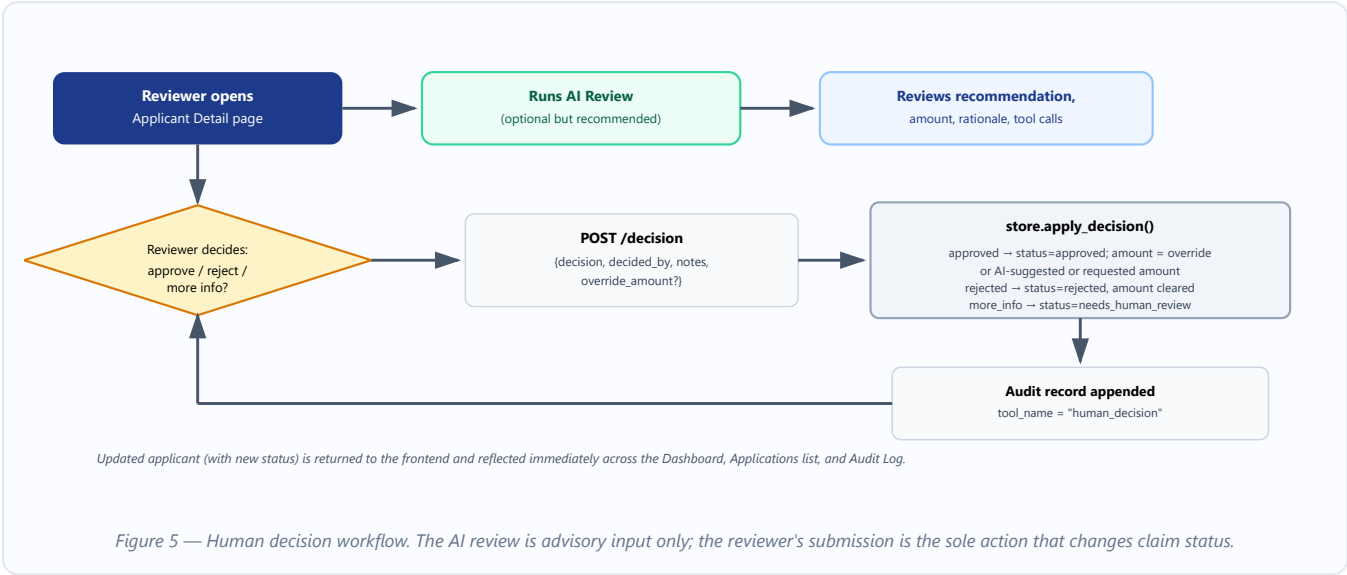
approval_fraction = clamp(1 - risk_penalty - prior_claims_penalty + tenure_bonus, 0, 1)
approved_amount   = round(requested_amount × approval_fraction / 100) × 100
```

6.5 Decision Derivation (decision_rules.derive_decision)



7. Human Decision Workflow

A reviewer's decision is submitted via `POST /api/applications/{id}/decision` (`store.apply_decision()`) — the **only** function in the codebase permitted to move a claim's status to `approved` or `rejected`.



7.1 Decision Outcomes

decision	Effect on <code>status</code>	Effect on <code>approved_amount</code>
<code>approved</code>	<code>approved</code>	<code>override_amount</code> if supplied, else the AI-recommended amount, else the full requested amount
<code>rejected</code>	<code>rejected</code>	cleared (<code>null</code>)
<code>more_info</code>	<code>needs_human_review</code>	unchanged

8. REST API Reference

Base URL: `http://localhost:8000/api` . Interactive Swagger docs are auto-generated by FastAPI at `/docs` .

Method & Path	Purpose	Response
GET <code>/health</code>	Liveness probe; also reports whether Claude is configured and which model is set.	<code>{status, claude_configured, model}</code>
GET <code>/applications</code>	List applicants; optional <code>status</code> , <code>sort_by</code> , <code>order</code> query params.	<code>Applicant[]</code>
GET <code>/applications/{id}</code>	Fetch one applicant. 404 if unknown.	<code>Applicant</code>
POST <code>/applications/{id}/ai-review</code>	Runs the agentic pipeline (Figure 2) for this applicant and persists the recommendation.	<code>AIRecommendation</code>
POST <code>/applications/{id}/decision</code>	Human reviewer submits a final decision (Figure 5).	<code>Applicant</code> (updated)
GET <code>/audit-log</code>	Full provenance trail; optional <code>applicant_id</code> filter.	<code>ToolCallRecord[]</code>
GET <code>/dashboard/summary</code>	Aggregated KPIs for the dashboard.	<code>DashboardSummary</code>

Error handling: POST `/ai-review` wraps the entire pipeline in a try/except at the route level; any unexpected failure returns HTTP 502 rather than a raw stack trace, since the pipeline already has its own internal fallbacks for the Claude-specific failure modes.

9. Data Model

All request/response shapes are defined once, as Pydantic models in `models/schemas.py`, and mirrored field-for-field by the TypeScript interfaces in `frontend/src/app/shared/models/api-models.ts`.

9.1 Applicant

Field	Type	Notes
applicant_id	string	e.g. APP-1001
name	string	
aadhar_number / aadhar_verified	string / bool	Masked format XXXX-XXXX-9999
pan_number / pan_verified	string / bool	Format AAAAA9999A
claim_type	enum	Hospitalization, Surgery, Outpatient, Critical Illness, Maternity, Accident
claim_amount_requested	number	₹
policy_tenure_years / prior_claims_count	int	
risk_score	number [0,1]	
status	enum	pending_review, approved, rejected, needs_human_review
approved_amount	number null	Set only via a human decision
ai_recommendation	AIRecommendation null	Latest AI review result, if any
submitted_at / last_updated	datetime	
decided_by / decision_notes	string null	Populated by the human decision endpoint

9.2 AIRecommendation

Field	Type	Notes
recommendation	enum	approve, reject, escalate
approved_amount	number null	From <code>calculate_risk_adjusted_amount</code> only
rationale	string	AI-written or templated prose
tool_calls	ToolCallRecord[]	Every tool invocation for this run
engine	enum	<code>claude</code> or <code>fallback_rules</code>
generated_at	datetime	

9.3 ToolCallRecord (audit entry)

Field	Type	Notes
id	int	Monotonically increasing
applicant_id / tool_name / timestamp	string / string / datetime	
inputs / outputs	object	Full function arguments and return value

10. Frontend — Application Structure

The Angular app uses standalone, lazily-loaded, route-level components with a shared component library. All backend I/O is centralized in a single injectable service.

10.1 Routes

Path	Component	Purpose
/dashboard	DashboardPage	KPI tiles + claims-by-status chart (default route)
/applications	ApplicationsPage	Filterable, sortable claims list
/applications/:id	ApplicantDetailPage	Profile, Run AI Review, decision form, provenance strip
/audit-log	AuditLogPage	Full tool-call & human-decision history

10.2 Shared Component Library (`shared/components/`)

- app-button
- app-card
- kpi-card
- status-badge
- verification-badge
- status-bar-chart
- page-hero
- gradient-heading
- skeleton
- empty-state
- top-nav

10.3 ApiService — the single I/O boundary

`shared/services/api.service.ts` is a thin, fully-typed wrapper around `HttpClient`. Every page injects this service; nothing in the UI calls the backend any other way, so the REST contract is documented in exactly one file.

```
getApplications(params)      → GET  /applications
getApplication(id)           → GET  /applications/{id}
runAiReview(id)              → POST /applications/{id}/ai-review
submitDecision(id, body)     → POST /applications/{id}/decision
getAuditLog(applicantId?)    → GET  /audit-log
getDashboardSummary()        → GET  /dashboard/summary
getHealth()                  → GET  /health
```

11. Audit Trail & Provenance

Every business-rule tool call — whether triggered by Claude's planner or the deterministic fallback — passes through the `@traced` decorator (`agent/provenance.py`), which records a `ToolCallRecord` with the tool name, exact inputs, exact outputs, and a UTC timestamp. Human decisions are logged the same way, under `tool_name = "human_decision"` . This unified log powers:

- The **Audit Log page**, filterable by applicant.
- The **provenance strip** on the Applicant Detail page, showing exactly which checks ran and what each one returned for the current AI recommendation.
- Regulatory / compliance traceability — every number a reviewer sees can be traced back to the tool call that produced it.

12. Configuration & Running the Application

12.1 Backend

```
cd backend
python -m venv .env
.venv\Scripts\activate
pip install -r requirements.txt
copy .env.example .env      # then fill in ANTHROPIC_API_KEY (optional)
uvicorn app.main:app --reload --port 8000
```

Interactive API docs: `http://localhost:8000/docs`

12.2 Frontend

```
cd frontend
npm install
ng serve
```

App available at `http://localhost:4200` , calling the backend at `http://localhost:8000/api` (configured in `shared/services/api-config.ts`).

12.3 Environment Variables (backend/.env)

Variable	Default	Purpose
ANTHROPIC_API_KEY	(empty)	If unset, the AI review pipeline automatically runs entirely on its deterministic fallback path — the app works fully either way.
CLAUDE_MODEL	<code>claude-sonnet-5</code>	Used for both the planner and rationale-writer calls.
CORS_ORIGINS	<code>http://localhost:4200,</code> <code>http://127.0.0.1:4200</code>	Comma-separated list of allowed frontend origins.

End of document — Claims Copilot Technical & Functional Documentation, v1.0.